

EXAMPLE TOUR

This document provides a quick guided tour of the Gurobi examples; we will try to highlight some of the most important features of these examples. Full source code is provided in this document, so you are free to explore the examples in full detail.

Wherever possible, we try to discuss the examples in a manner that is independent of programming languages. We will refer to each example using a brief, language independent name. You will need to map this name to the specific source file name for your language. For example, the `facility` example corresponds to six different implementations, one in C (*facility_c.c*), one in C++ (*facility_c++.cpp*), one in Java (*Facility.java*), one in C# (*facility_cs.cs*), one in Visual Basic (*facility_vb.vb*), and one in Python (*facility.py*). If you would like to look at the language implementation for a particular example, please refer to the appropriate example source file.

1.1 Topics covered in the examples

The easiest place to start your introduction to the Gurobi examples is probably with the examples that *load and solve a model from a file*. These demonstrate the most basic capabilities of the Gurobi libraries. They also demonstrate the use of model attributes, which are an important concept in the Gurobi Optimizer.

Once you are comfortable with these examples, you should move on to the examples that *build a model* from scratch. These show you how to create variables and constraints, and add them to an optimization model.

The next topic covered in this document is *model modification*. The Gurobi distribution includes examples that add and remove constraints, add variables, and change variable types, bounds and objective coefficients. You modify a model in much the same way that you build a model from scratch, but there are some important differences involving the use of the solution information.

Next, this document covers *parameter changes*. The `params` example shows you how to change parameters, and in particular how to use different parameter settings for different models.

The *infeasibility* section considers a few examples that cope with model infeasibility. Some use an Irreducible Inconsistent Subsystem (IIS) to handle the infeasibility, while others relax constraints.

One useful MIP feature that is worth understanding is *MIP starts*. A MIP start allows you to specify a known feasible solution to the MIP solver. The solution provides a bound on the objective of the best possible solution, which can help to limit the MIP search. The solution also provides a potential start point for the local search heuristics that are utilized by the Gurobi MIP solver.

It is possible to achieve model-data separation when using our Python interface, as is often done in modeling languages, but you need to make use of Python modules to do so. The *model-data separation* section provides an example of how this is done. It considers three versions of the diet example. All three use the same function to formulate and solve the actual optimization model, but they obtain model data from very different places.

The final topic we cover in this document is *Gurobi callbacks*. Callbacks allow the user to obtain periodic progress information related to the optimization.

1.1.1 A list of the Gurobi examples

We recommend that you begin by reading the overview of the examples (which begins in the *next section*). However, if you'd like to dive directly into a specific example, the following is a list of all of the examples included in the Gurobi distribution, organized by basic function. The source for the examples can be found by following the provided links, or in the `examples` directory of the Gurobi distribution.

Read a model from a file

Example	Description	Available Languages
<i>lp</i>	A very simple example that reads a continuous model from a file, optimizes it, and writes the solution to a file. If the model is infeasible, it writes an Irreducible Inconsistent Subsystem (IIS) instead.	<i>C, C++, C#, Java, Python, R, Visual Basic</i>
<i>mip2</i>	Reads a MIP model from a file, optimizes it, and then solves the fixed version of the MIP model.	<i>C, C++, C#, Java, Python, Visual Basic</i>

Build a simple model

Example	Description	Available Languages
<i>mip1</i>	Builds a trivial MIP model, solves it, and prints the solution.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>qp</i>	Builds a trivial QP model, solves it, converts it to an MIQP model, and solves it again.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>qcp</i>	Builds and solves a trivial QCP model.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>bilinear</i>	Builds and solves a trivial bilinear model.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>sos</i>	Builds and solves a trivial SOS model.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>dense</i>	Solves a model stored using dense matrices. We don't recommend using dense matrices, but this example may be helpful if your data is already in this format.	<i>C, C++, C#, Java, Python, Visual Basic</i>
<i>genconstr</i>	Demonstrates the use of simple general constraints.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>genconstrnl</i>	Demonstrates the use of general nonlinear constraints.	<i>C, C++, C#, Java, Python, Visual Basic</i>
<i>matrix1</i>	A Python-only example that formulates and solves a simple MIP model using the matrix API.	<i>Python</i>
<i>multiobj</i>	Demonstrates the use of multi-objective optimization.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>piecewise</i>	Demonstrates the use of piecewise-linear objective functions.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>gc_pwl</i>	Demonstrates the use of piecewise-linear constraint.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>gc_pwl_fun</i>	Demonstrates the use of function constraints.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>gc_funcno1</i>	Builds and solves a simple nonlinear model.	<i>C, C++, C#, Java, MATLAB, Python, R, VB</i>
<i>poolsearch</i>	Demonstrates the use of solution pools.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>

A few simple applications

Example	Description	Available Languages
<i>ACOPF</i>	A Python-only example that solves a simple ACOPF problem on four buses. It demonstrates the use of several Python modeling constructs, including vectorized constraints and nonlinear expressions.	<i>Python</i>
<i>diet</i>	Builds and solves the classic diet problem. Demonstrates model construction and simple model modification - after the initial model is solved, a constraint is added to limit the number of dairy servings.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>diet2, diet3, diet4, dietmodel</i>	Python-only variants of the diet example that illustrate model-data separation.	<i>Python</i>
<i>facility</i>	Simple facility location model: given a set of plants and a set of warehouses, with transportation costs between them, this example finds the least expensive set of plants to open in order to satisfy product demand. This example demonstrates the use of MIP starts — the example computes an initial, heuristic solution and passes that solution to the MIP solver.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>matrix2</i>	Python-only example that solves the n-queens problem using the matrix-oriented Python interface.	<i>Python</i>
<i>netflow</i>	A Python-only example that solves a multi-commodity network flow model. It demonstrates the use of several Python modeling constructs, including dictionaries, tuples, tupledict, and tuplelist objects.	<i>Python</i>
<i>portfolio</i>	A Python-only example that solves a financial portfolio optimization model, where the historical return data is stored using the pandas package and the result is plotted using the matplotlib package. It demonstrates the use of pandas, NumPy, and Matplotlib in conjunction with Gurobi.	<i>Python</i>
<i>sudoku</i>	Reads a Sudoku puzzle data set from a file, builds a MIP model to solve that model, solves it, and prints the solution.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>workforce1</i>	Formulates and solves a workforce scheduling model. If the model is infeasible, the example computes and prints an Irreducible Inconsistent Subsystem (IIS).	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>workforce2</i>	An enhancement of <i>workforce1</i> . This example solves the same workforce scheduling model, but if the model is infeasible, it computes an IIS, removes one of the associated constraints from the model, and re-solves. This process is repeated until the model becomes feasible. Demonstrates constraint removal.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>workforce3</i>	A different enhancement of <i>workforce1</i> . This example solves the same workforce scheduling model, but if the model is infeasible, it adds artificial variables to each constraint and minimizes the sum of the artificial variables. This corresponds to finding the minimum total change in the right-hand side vector required in order to make the model feasible. Demonstrates variable addition.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>workforce4</i>	An enhancement of <i>workforce3</i> . This example solves the same workforce scheduling model, but it starts with artificial variables in each constraint. It first minimizes the sum of the artificial variables. Then, it introduces a new quadratic objective to balance the workload among the workers. Demonstrates optimization with multiple objective functions.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>workforce5</i>	An alternative enhancement of <i>workforce3</i> . This example solves the same workforce scheduling model, but it starts with artificial variables in each constraint. It formulates a multi-objective model where the primary objective is to minimize the sum of the artificial variables (uncovered shifts), and the secondary objective is to minimize the maximum difference in the number of shifts worked between any pair of workers. Demonstrates multi-objective optimization.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>

1.1. Topics covered in the examples

Illustrating specific features

Example	Description	Available Languages
<i>feasopt</i>	Reads a MIP model from a file, adds artificial slack variables to relax each constraint, and then minimizes the sum of the artificial variables. It then computes the same relaxation using the <i>feasibility relaxation</i> feature. The example demonstrates simple model modification by adding slack variables. It also demonstrates the feasibility relaxation feature.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>lp-method</i>	Demonstrates the use of different LP algorithms. Reads a continuous model from a file and solves it using multiple algorithms, reporting which is the quickest for that model.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>lpmod</i>	Demonstrates the use of advanced starts in LP. Reads a continuous model from a file, solves it, and then modifies one variable bound. The resulting model is then solved in two different ways: starting from the solution of the original model, or restarting from scratch.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>params</i>	Demonstrates the use of Gurobi parameters. Reads a MIP model from a file, and then spends 5 seconds solving the model with each of four different values of the <i>MIPFocus</i> parameter. It compares the optimality gaps for the four different runs, and continues with the <i>MIPFocus</i> value that produced the smallest gap.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>sensitivity</i>	MIP sensitivity analysis. Reads a MIP model, solves it, and then computes the objective impact of fixing each binary variable in the model to 0 or 1. Demonstrates the multi-scenario feature.	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>tune</i>	Uses the parameter tuning tool to search for improved parameter settings for a model.	<i>C, C++, C#, Java, Python, Visual Basic</i>
<i>fix-and-divide</i>	Implements a simple MIP heuristic. It reads a MIP model from a file, relaxes the integrality conditions, and then solves the relaxation. It then chooses a set of integer variables that take integer or nearly integer values in the relaxation, fixes them to the nearest integer, and solves the relaxation again. This process is repeated until the relaxation is either integer feasible or linearly infeasible. The example demonstrates different types of model modification (relaxing integrality conditions, changing variable bounds, etc.).	<i>C, C++, C#, Java, MATLAB, Python, R, Visual Basic</i>
<i>multiscenario</i>	Simple facility location model: given a set of plants and a set of warehouses, with transportation costs between them, this example finds the least expensive set of plants to open in order to satisfy product demand. Since the plant fixed costs and the warehouse demands are uncertain, multiple scenarios are created to capture different possible values. A multi-scenario model is constructed and solved, and the solutions for the different scenarios are retrieved and displayed.	<i>C, C++, C#, Java, Python, Visual Basic</i>
<i>batch-mode</i>	Demonstrates the use of batch optimization.	<i>C, C++, C#, Java, Python, Visual Basic</i>
<i>workforce_batch</i>	Example which formulates a workforce scheduling model. The model is solved using batch optimization. The <i>VTag</i> attribute is used to identify the set of variables whose solution information is needed to construct the schedule.	<i>Python</i>
<i>mip1_remodel</i>	Example that shows the use of context managers to create and dispose of environment and model objects.	<i>Python</i>

More advanced features

Example	Description	Available Languages
<i>tsp</i>	Solves a traveling salesman problem using lazy constraints.	<i>C, C++, C#, Java, Python, Visual Basic</i>
<i>callback</i>	Demonstrates the use of Gurobi callbacks.	<i>C, C++, C#, Java, Python, Visual Basic</i>

1.1.2 Load and solve a model from a file

Examples

batchmode, callback, feaso, fixanddive, lp, lpmethod, lpm, mip2, params, sensitivity

One of the most basic tasks you can perform with the Gurobi libraries is to read a model from a file, optimize it, and report the result. The *lp example* is a simple illustration of how this is done in the various supported Gurobi languages. While the specifics vary from one language to another, the basic structure remains the same for all languages.

After initializing the Gurobi environment, the examples begin by reading the model from the specified file.

C

C++

C#

Java

Python

Visual Basic

```
error = GRBreadmodel(env, argv[1], &model);
if (error) goto QUIT;
```

```
GRBModel model = GRBModel(env, argv[1]);
```

```
GRBModel model = new GRBModel(env, args[0]);
```

```
GRBModel model = new GRBModel(env, args[0]);
```

```
model = gp.read(sys.argv[1])
```

```
Dim model As GRBModel = New GRBModel(env, args(0))
```

The next step is to invoke the Gurobi Optimizer on the model.

C

C++

C#

Java

Python

Visual Basic

```
error = GRBoptimize(model);  
if (error) goto QUIT;
```

```
model.optimize();
```

```
model.Optimize();
```

```
model.optimize();
```

```
model.optimize()
```

```
model.Optimize()
```

A successful `optimize` call populates a set of solution attributes in the model. For example, once the call completes, the `X` variable attribute contains the solution value for each variable. Similarly, for continuous models, the `Pi` constraint attribute contains the dual value for each constraint.

The examples then retrieve the value of the model `Status` attribute to determine the result of the optimization. In the *lp example*, an optimal solution is written to a solution file (`model.sol`).

There are many other things you can do once you have read and solved the model. For example, *lp example* checks the solution status – which is highly recommended. If the model is found to be infeasible, this example computes an Irreducible Inconsistent Subsystem (IIS) to isolate the source of the infeasibility.

1.1.3 Build a model

Examples

bilinear, diet, facility, gc_pwl, gc_pwl_func, genconstr, matrix1, mip1, multiobj, multiscenario, piecewise, poolsearch, qcp, qp, sensitivity, sos, sudoku, workforce1, workforce_batchmode, workforce2, workforce3, workforce4, workforce5

Several of the Gurobi examples build models from scratch. We start by focusing on two: *mip1 example* and *sos example*. Both build very simple models to illustrate the basic process.

Typically, the first step in building a model is to create an empty model.

C

C++

C#

Java

Python

Visual Basic


```
error = GRBnewmodel(env, &model, "mip1", 0, NULL, NULL, NULL, NULL, NULL);
if (error) goto QUIT;
```

You can optionally create a set of variables when you create the model, as well as specifying bounds, objective coefficients, and names for these variables. These examples add new variables separately.

```
GRBModel model = new GRBModel(env);
```

```
GRBModel model = new GRBModel(env);
```

```
GRBModel model = new GRBModel(env);
```

```
m = gp.Model("mip1")
```

```
Dim model As GRBModel = New GRBModel(env)
```

Once the model has been created, the typical next step is to add variables.

C

C++

C#

Java

Python

Visual Basic

```
error = GRBaddvars(model, 3, 0, NULL, NULL, NULL, obj, NULL, NULL, vtype, NULL);
if (error) goto QUIT;
```

```
GRBVar x = model.addVar(0.0, 1.0, 0.0, GRB.BINARY, "x");
```

```
GRBVar x = model.AddVar(0.0, 1.0, 0.0, GRB.BINARY, "x");
```

```
GRBVar x = model.addVar(0.0, 1.0, 0.0, GRB.BINARY, "x");
```

```
GRBVar x = model.addVar(0.0, 1.0, 0.0, GRB.BINARY, "x")
```

```
Dim x As GRBVar = model.AddVar(0.0, 1.0, 0.0, GRB.BINARY, "x")
```

The new variable's lower bound, upper bound, objective coefficient, type, and name are specified as arguments. In C++ and Python, you can omit these arguments and use default values; see the [Gurobi Reference Manual](#) for details.

The next step is to add (linear) constraints to the model.

C

C++

C#

Java

Python

Visual Basic

```
error = GRBaddconstr(model, 3, ind, val, GRB_LESS_EQUAL, 4.0, "c0");  
if (error) goto QUIT;
```

To add a linear constraint in C, you must specify a list of variable indices and coefficients for the left-hand side, a sense for the constraint (e.g., `GRB_LESS_EQUAL`), and a right-hand side constant. You can also give the constraint a name; if you omit the name, Gurobi will assign a default name for the constraint.

In C++ you build a linear constraint by first building linear expressions for the left- and right-hand sides. For C++ the standard mathematical operators such as `+`, `*`, `<=` have been overloaded so that the linear expression resembles a traditional mathematical expression.

```
model.addConstr(x + 2 * y + 3 * z <= 4.0, "c0");
```

In C# you build a linear constraint by first building linear expressions for the left- and right-hand sides. For C# the standard mathematical operators such as `+`, `*`, `<=` have been overloaded so that the linear expression resembles a traditional mathematical expression.

```
model.AddConstr(x + 2 * y + 3 * z <= 4.0, "c0");
```

In Java you build a linear constraint by first building linear expressions for the left- and right-hand sides. In Java, which doesn't support operator overloading, you build an expression as follows:

```
GRBLinExpr expr = new GRBLinExpr();  
expr.addTerm(1.0, x); expr.addTerm(2.0, y); expr.addTerm(3.0, z);
```

You then use the `addConstr` method on the `GRBModel` object to add a constraint using these linear expressions for the left- and right-hand sides:

```
model.addConstr(expr, GRB.LESS_EQUAL, 4.0, "c0");
```

In Python you build a linear constraint by first building linear expressions for the left- and right-hand sides. For Python the standard mathematical operators such as `+`, `*`, `<=` have been overloaded so that the linear expression resembles a traditional mathematical expression.

```
model.addConstr(x + 2 * y + 3 * z <= 4.0, "c0")
```

In Visual Basic you build a linear constraint by first building linear expressions for the left- and right-hand sides. For Python the standard mathematical operators such as `+`, `*`, `<=` have been overloaded so that the linear expression resembles a traditional mathematical expression.

```
model.AddConstr(x + 2 * y + 3 * z <= 4.0, "c0")
```

Once the model has been built, the typical next step is to optimize it (using `GRBoptimize()` in C, `model.optimize` in C++, Java, and Python, or `model.Optimize` in C#). You can then query the `X` attribute on the variables to retrieve the solution (and the `VarName` attribute to retrieve the variable name for each variable).

C

C++

C#

Java

Python

Visual Basic

```
error = GRBgetdblattrarray(model, GRB_DBL_ATTR_X, 0, 3, sol);
if (error) goto QUIT;
```

```
cout << x.get(GRB_StringAttr_VarName) << " " << x.get(GRB_DoubleAttr_X) << endl;
cout << y.get(GRB_StringAttr_VarName) << " " << y.get(GRB_DoubleAttr_X) << endl;
cout << z.get(GRB_StringAttr_VarName) << " " << z.get(GRB_DoubleAttr_X) << endl;
```

```
Console.WriteLine(x.VarName + " " + x.X);
Console.WriteLine(y.VarName + " " + y.X);
Console.WriteLine(z.VarName + " " + z.X);
```

```
System.out.println(x.get(GRB.StringAttr.VarName) + " " + x.get(GRB.DoubleAttr.X));
System.out.println(y.get(GRB.StringAttr.VarName) + " " + y.get(GRB.DoubleAttr.X));
System.out.println(z.get(GRB.StringAttr.VarName) + " " + z.get(GRB.DoubleAttr.X));
```

```
for v in m.getVars():
    print(f"{v.VarName} {v.X:g}")
```

```
Console.WriteLine(x.VarName & " " & x.X)
Console.WriteLine(y.VarName & " " & y.X)
Console.WriteLine(z.VarName & " " & z.X)
```

When querying or modifying attribute values for an array of constraints or variables, it is generally more efficient to perform the action on the whole array at once. This is quite natural in the C interface, where most of the attribute routines take array arguments. In the C++, C#, Java, and Python interfaces, you can use the `get` and `set` methods on the `GRBModel` object to work directly with arrays of attribute values (`getAttr/getAttr` in Python).

C

C++

C#

Java

Python

Visual Basic

In the *mip1*, C example, this is done as follows:

```
error = GRBgetdblattrarray(model, GRB_DBL_ATTR_X, 0, 3, sol);
if (error) goto QUIT;
```

In the *sensitivity* C++ example, this is done as follows:

```
origX = model.get(GRB_DoubleAttr_X, vars, numVars);
```

In the *sudoku* C# example, this is done as follows:

```
double[,] x = model.Get(GRB.DoubleAttr.X, vars);
```

In the *sudoku* Java example, this is done as follows:

```
double[][][] x = model.get(GRB.DoubleAttr.X, vars);
```

In the *sudoku* Python example, this is done as follows:

```
solution = model.getAttr('X', vars)
```

In the *sudoku* Visual Basic example, this is done as follows:

```
Dim x As Double(,,) = model.Get(GRB.DoubleAttr.X, vars)
```

Important: We should point out one important subtlety in our interface. We use a *lazy update* approach to building and modifying a model. When you make changes, they are added to a queue. The queue is only flushed when you optimize the model (or write it to a file). In the uncommon situation where you want to query information about your model before optimizing it, you should call the *update* method before making your query.

1.1.4 Additional modeling elements

Examples

bilinear, gc_pwl, gc_pwl_func, genconstr, multiobj, multiscenario, piecewise, qcp, qp, sensitivity, sos

A mathematical programming model in its traditional form consists of a linear objective, a set of linear constraints, and a set of continuous or integer decision variables. Gurobi supports a number of additional modeling constructs. In addition to linear constraints, Gurobi also supports SOS constraints, quadratic constraints, and general constraints. In addition to a single linear objective, Gurobi also supports quadratic objectives, piecewise-linear objectives, and multiple linear objectives. Consult the corresponding examples from the Gurobi distribution for simple examples of how to use each of these modeling elements.

1.1.5 Modify a model

Examples

diet, feaso, fixanddivide, gc_pwl_func, lpmod, sensitivity, workforce3, workforce4, workforce5

This section considers model modification. Modification can take many forms, including adding constraints or variables, deleting constraints or variables, modifying constraint and variable attributes, changing constraint coefficients, etc. The Gurobi examples don't cover all possible modifications, but they cover the most common types.

Diet example

This example builds a linear model that solves the classic diet problem: to find the minimum cost diet that satisfies a set of daily nutritional requirements. Once the model has been formulated and solved, it adds an additional constraint to limit the number of servings of dairy products and solves the model again. Let's focus on the model modification.

Adding constraints to a model that has already been solved is no different from adding constraints when constructing an initial model. In the different APIs, we can introduce a limit of 6 dairy servings through the following constraint (where variables 6 and 7 capture the number of servings of milk and ice cream, respectively):

C

C++

C#

Java

Python

Visual Basic

```
printf("\nAdding constraint: at most 6 servings of dairy\n");
cind[0] = 7;
cval[0] = 1.0;
cind[1] = 8;
cval[1] = 1.0;
error = GRBaddconstr(model, 2, cind, cval, GRB_LESS_EQUAL, 6.0, "limit_dairy");
if (error) goto QUIT;
```

```
cout << "\nAdding constraint: at most 6 servings of dairy" << endl;
model.addConstr(buy[7] + buy[8] <= 6.0, "limit_dairy");
```

```
Console.WriteLine("\nAdding constraint: at most 6 servings of dairy");
model.AddConstr(buy[7] + buy[8] <= 6.0, "limit_dairy");
```

```
lhs.addTerm(1.0, buy[8]);
model.addConstr(lhs, GRB.LESS_EQUAL, 6.0, "limit_dairy");
```

```
print("\nAdding constraint: at most 6 servings of dairy")
m.addConstr(buy.sum(["milk", "ice cream"]) <= 6, "limit_dairy")
```

```
Console.WriteLine(vbLf & "Adding constraint: at most 6 servings of dairy")
model.AddConstr(buy(7) + buy(8) <= 6, "limit_dairy")
```

For linear models, the previously computed solution can be used as an efficient *warm start* for the modified model. The Gurobi solver retains the previous solution, so the next `optimize` call automatically starts from the previous solution.

Lpmod example

Changing a variable bound is also straightforward. The *lpmod example* changes a single variable bound, then re-solves the model in two different ways. A variable bound can be changed by modifying the UB or LB attribute of the variable.

C

C++

C#

Java

Python

Visual Basic

```
error = GRBsetdblattrelement(model, "UB", minVar, 0.0);
if (erro) goto QUIT;
```

```
v[minVar].set(GRB_DoubleAttr_UB, 0.0);
```

```
minVar.UB = 0.0;
```

```
minVar.set(GRB.DoubleAttr.UB, 0.0);
```

```
minVar.UB = 0.0
```

```
minVar.UB = 0
```

The model is re-solved simply by calling the `optimize` method again. For a continuous model, this starts the optimization from the previous solution. To illustrate the difference when solving the model from an initial, unsolved state, the *lpmod example* calls the `reset` function.

C

C++

C#

Java

Python

Visual Basic

```
error = GRBreset(model, 0);  
if (error) goto QUIT;
```

```
model.reset();
```

```
model.Reset();
```

```
model.reset();
```

```
model.reset();
```

```
model.Reset()
```

When we call the `optimize` method after resetting the model, optimization starts from scratch. Although the difference in computation time is insignificant for this tiny example, a warm start can make a big difference for larger models.

Fix-and-divide example

The *fixanddivide example* provides another example of bound modification. In this case, we repeatedly modify a set of variable bounds, utilizing warm starts each time. In the different APIs, variables are fixed as follows:

C

C++

C#

Java

Python

Visual Basic

```

for (j = 0; j < nfix; ++j)
{
    fixval = floor(fractional[j].X + 0.5);
    error = GRBsetdblattr(element(model, "LB", fractional[j].index, fixval);
    if (error) goto QUIT;
    error = GRBsetdblattr(element(model, "UB", fractional[j].index, fixval);
    if (error) goto QUIT;
    error = GRBgetstrattr(element(model, "VarName", fractional[j].index, &vname);
    if (error) goto QUIT;
    printf(" Fix %s to %f ( rel %f )\n", vname, fixval, fractional[j].X);
}

```

```

for (int i = 0; i < nfix; ++i)
{
    GRBVar* v = fractional[i];
    double fixval = floor(v->get(GRB_DoubleAttr_X) + 0.5);
    v->set(GRB_DoubleAttr_LB, fixval);
    v->set(GRB_DoubleAttr_UB, fixval);
    cout << " Fix " << v->get(GRB_StringAttr_VarName) << " to " <<
    fixval << " ( rel " << v->get(GRB_DoubleAttr_X) << " )" <<
    endl;
}

```

```

for (int i = 0; i < nfix; ++i) {
    GRBVar v = fractional[i];
    double fixval = Math.Floor(v.X + 0.5);
    v.LB = fixval;
    v.UB = fixval;
    Console.WriteLine(" Fix " + v.VarName +
        " to " + fixval + " ( rel " + v.X + " )");
}

```

```

for (int i = 0; i < nfix; ++i) {
    GRBVar v = fractional.get(i);
    double fixval = Math.floor(v.get(GRB.DoubleAttr.X) + 0.5);
    v.set(GRB.DoubleAttr.LB, fixval);
    v.set(GRB.DoubleAttr.UB, fixval);
    System.out.println(" Fix " + v.get(GRB.StringAttr.VarName) +
        " to " + fixval + " ( rel " + v.get(GRB.DoubleAttr.X) + " )");
}

```

```

for i in range(nfix):
    v = fractional[i]
    fixval = int(v.X + 0.5)
    v.LB = fixval
    v.UB = fixval
    print(f" Fix {v.VarName} to {fixval:g} (rel {v.X:g})")

```

```

Dim nfix As Integer = Math.Max(fractional.Count / 4, 1)
For i As Integer = 0 To nfix - 1

```

(continues on next page)

(continued from previous page)

```

Dim v As GRBVar = fractional(i)
Dim fixval As Double = Math.Floor(v.X + 0.5)
v.LB = fixval
v.UB = fixval
Console.WriteLine("  Fix " & v.VarName & " to " & fixval & _
                  " ( rel " & v.X & " )")
Next

```

Again, the subsequent call to `optimize` starts from the previous solution.

Sensitivity analysis example

The *sensitivity example* computes the optimal objective value associated with fixing each binary variable to 0 or 1. It first solves the given model to optimality. It then constructs a multi-scenario model, where in each scenario a binary variable is fixed to the complement of the value it took in the optimal solution. The resulting multi-scenario model is solved, giving the objective degradation associated with forcing each binary variable off of its optimal value.

Feasopt example

The last modification example we consider is the *feasopt example*, which adds variables to existing constraints and also changes the optimization objective. Setting the objective to zero is straightforward. In C, set the `Obj` attribute to 0. In the object-oriented interfaces, call `setObjective` with an empty linear expression.

C

C++

C#

Java

Python

Visual Basic

```

for (j = 0; j < numvars; ++j)
{
    error = GRBsetdblattrelement(model, "Obj", j, 0.0);
    if (error) goto QUIT;
}

```

```
feasmodel.setObjective(GRBLinExpr(0.0));
```

```
feasmodel.SetObjective(new GRBLinExpr());
```

```
feasmodel.setObjective(new GRBLinExpr());
```

```
feasmodel.setObjective(0.0)
```

```
feasmodel.SetObjective(New GRBLinExpr())
```


Adding new variables is somewhat more complex. In the example, we want to add artificial variable(s) to each constraint in order to allow the constraint to be relaxed. We use two artificial variables for equality constraints and one for inequality constraints.

C

C++

C#

Java

Python

Visual Basic

```
error = GRBgetstrattrelement(model, "ConstrName", i, &cname);
if (error) goto QUIT;
vname = malloc(sizeof(char) * (6 + strlen(cname)));
if (!vname) goto QUIT;
strcpy(vname, "ArtN_");
strcat(vname, cname);
vind[0] = i;
vval[0] = -1.0;
error = GRBaddvar(model, 1, vind, vval, 1.0, 0.0, GRB_INFINITY, GRB_CONTINUOUS, vname);
if (error) goto QUIT;
```

```
double coef = -1.0;
feasmodel.addVar(0.0, GRB_INFINITY, 1.0, GRB_CONTINUOUS, 1,
                &c[i], &coef, "ArtN_" + c[i].get(GRB_StringAttr_ConstrName));
```

```
GRBConstr[] constrs = new GRBConstr[] { c[i] };
double[] coeffs = new double[] { -1 };
feasmodel.AddVar(0.0, GRB.INFINITY, 1.0, GRB.CONTINUOUS, constrs,
                coeffs, "ArtN_" + c[i].ConstrName);
```

```
GRBConstr[] constrs = new GRBConstr[] { c[i] };
double[] coeffs = new double[] { -1 };
feasmodel.addVar(0.0, GRB.INFINITY, 1.0, GRB.CONTINUOUS, constrs,
                coeffs, "ArtN_" + c[i].get(GRB.StringAttr_ConstrName));
```

```
feasmodel.addVar(
    obj=1.0, name=f"ArtN_{c.ConstrName}", column=gp.Column([-1], [c])
)
```

We use the `column` argument of the `addVar` method to specify the set of constraints in which the new variable participates, as well as the associated coefficients. In this example, the new variable only participates in the constraint to be relaxed. Default values are used here for all variables attributes except the objective and the variable name.

```
Dim constrs As GRBConstr() = New GRBConstr() {c(i)}
Dim coeffs As Double() = New Double() {-1}
feasmodel.AddVar(0.0, GRB.INFINITY, 1.0, GRB.CONTINUOUS, _
                constrs, coeffs, _
                "ArtN_" & c(i).ConstrName)
```

1.1.6 Change parameters

Examples

batchmode, *callback*, *fixanddive*, *gc_pwl_func*, *lp*, *lpmethod*, *mip2*, *multiscenario*, *params*, *sensitivity*, *workforce_batchmode*

This section illustrates the use of Gurobi parameters. *Example params* reads a MIP model from a file, then solves the model using four different values of the [MIPFocus](#) parameter, running for five seconds per value ([MIPFocus](#) chooses the high-level strategy that the MIP solver uses to solve the problem). It then chooses the parameter value that produced the smallest MIP gap, and continues solving the model until it achieves optimality.

The mechanics of setting a parameter are quite simple. To set the [MIPFocus](#) parameter in different APIs, do the following:

C

C++

C#

Java

Python

Visual Basic

```
error = GRBsetintparam(modelenv, "MIPFocus", i);  
if (error) goto QUIT;
```

```
m->set(GRB_IntParam_MIPFocus, i);
```

```
m.Parameters.MIPFocus = i;
```

```
m.set(GRB.IntParam.MIPFocus, i);
```

```
m.Params.MIPFocus = i
```

```
m.Parameters.MIPFocus = i
```

Note: We should add one quick comment about how parameter settings propagate between different models. When we set the [TimeLimit](#) parameter on a model, then make a copy of that model, the parameter setting is carried over to the copy. When we set the [MIPFocus](#) parameter on the copy, that parameter change has no effect on the other copies, nor on the original model.

1.1.7 Diagnose and cope with infeasibility

Examples

feasopt, *lp*, *multiscenario*, *sensitivity*, *workforce1*, *workforce2*, *workforce3*

When solving optimization models, there are some situations where the specified constraints cannot be satisfied. When this happens, you often need to either identify and repair the root cause of the infeasibility, or alternatively find a set of constraints to relax in order to obtain a feasible model. The *workforce1 example*, *workforce2 example*, and *workforce3 example* illustrate these different strategies.

Starting with the simplest of the three examples, *workforce1 example* formulates a simple workforce scheduling model and solves it. If the model is infeasible, it computes an Irreducible Inconsistent Subsystem (IIS). The user can then inspect this information to understand and hopefully address the source of the infeasibility in the model.

Example workforce2 is similar, except that if the model is infeasible, the example repeatedly identifies an IIS and removes one of the associated constraints from the model until the model becomes feasible. Note that it is sufficient to remove one constraint from the IIS to address that source of infeasibility, but that one IIS may not capture all sources of infeasibility. It is therefore necessary to repeat the process until the model is feasible.

Example workforce3 takes a different approach to addressing infeasibility. Rather than identifying and removing IIS members, it allows the constraints of the model to be relaxed. Like the *feasopt example*, an artificial variable is added to each constraint. The example sets the objective on the original variables to zero, and then solves a model that minimizes the total magnitude of the constraint relaxation.

The *feasopt example* demonstrates another approach to relaxing an infeasible model. It computes a *feasibility relaxation* for the infeasible model. A feasibility relaxation is a model that, when solved, minimizes the amount by which the solution violates the bounds and linear constraints of the original model. This method is invoked as follows:

C

C++

C#

Java

Python

Visual Basic

```
error = GRBfeasrelax(feasmodel, GRB_FEASRELAX_LINEAR, 1, NULL, NULL, rhspen, &feasobj);
if (error) goto QUIT;
```

```
feasmodel1.fasRelax(GRB_FEASRELAX_LINEAR, true, false, true);
```

```
feasmodel1.FasRelax(GRB.FEASRELAX_LINEAR, true, false, true);
```

```
feasmodel1.fasRelax(GRB.FEASRELAX_LINEAR, true, false, true);
```

```
feasmodel1.fasRelaxS(0, True, False, True)
```

```
feasmodel1.FasRelax(GRB.FEASRELAX_LINEAR, true, false, true)
```

The arguments to this method select the objective function for the relaxed model, the specific set of bounds and constraints that are allowed to be relaxed, and the penalties for relaxing specific bounds and constraints.

1.1.8 MIP starts

Examples

facility, sensitivity

A MIP modeler often knows how to compute a feasible solution to their problem. In cases where the MIP solver is slow in finding an initial feasible solution, it can be helpful for the modeler to provide a feasible solution along with the model itself. This is done through the [Start](#) attribute on the variables. This is illustrated in the *facility examples*.

The *facility examples* solves a simple facility location problem. The model contains a set of warehouses, and a set of plants that produce the products required in the warehouses. Each plant has a maximum production capacity and a fixed operating cost. Additionally, there is a cost associated with shipping products from a plant to a warehouse. The goal is to decide which plants should satisfy the demand for the product, given the associated capacities and costs.

The example uses a simple heuristic for choosing an initial solution: it closes the plant with the highest fixed cost. The associated solution may not be optimal, but it could produce a reasonable starting solution for the MIP optimization. The MIP start is passed to the MIP solver by setting the [Start](#) attribute before the optimization begins. In the different APIs, we set the start attribute to open all plants using the following code:

C

C++

C#

Java

Python

Visual Basic

```
/* First, open all plants */
for (p = 0; p < nPlants; ++p)
{
    error = GRBsetdblattr(model, "Start", opencol(p), 1.0);
    if (error) goto QUIT;
}
```

```
// First, open all plants
for (p = 0; p < nPlants; ++p)
{
    open[p].set(GRB_DoubleAttr_Start, 1.0);
}
```

```
// First, open all plants
for (int p = 0; p < nPlants; ++p) {
    open[p].Start = 1.0;
}
```

```
// First, open all plants
for (int p = 0; p < nPlants; ++p) {
    open[p].set(GRB.DoubleAttr.Start, 1.0);
}
```

```
# First open all plants
for p in plants:
    open[p].Start = 1.0
```

```
' First, open all plants
For p As Integer = 0 To nPlants - 1
    open(p).Start = 1.0
Next
```

When you run the example, the MIP solver reports that the start produced a feasible initial solution:

```
User MIP start produced solution with objective 210500 (0.01s)
Loaded user MIP start with objective 210500
```

This initial solution turns out to be optimal for the sample data. Although the computation difference is insignificant for this tiny example, providing a good starting solution can sometimes help for more difficult models.

Note: The MIP start in this example only specifies values for some of the variables – the variables that determine which plants to leave open and which plants to close. The Gurobi MIP solve uses whatever start information is provided to try to construct a complete solution.

1.1.9 Model-data separation in Python

Examples (Python only)

diet2, diet3, diet4

When building an optimization model in a modeling language, it is typical to separate the optimization model itself from the data used to create an instance of the model. These two model ingredients are often stored in completely different files. We show how a similar result can be achieved in our Python interface with our *diet2 example*, *diet3 example* and *diet4 example*. These examples illustrate alternate approaches to providing data to the optimization model: *diet2 example* embeds the data in the source file, *diet3 example* reads the data from an SQL database (using the Python `sqlite3` package), and *diet4 example* reads the data from an Excel spreadsheet (using the Python `xlrd` package). *dietmodel.py* contains the optimization model itself. The same model is used by *diet2 example*, *diet3 example* and *diet4 example*.

The key construct that enables the separation of the model from the data is the Python module. A module is simply a set of functions and variables, stored in a file. You import a module into a program using the `import` statement. *diet2 example*, *diet3 example* and *diet4 example* all populate a set of variables, and then pass them to the `solve` function of the *dietmodel module* using the following pair of statements:

```
import dietmodel
dietmodel.solve(categories, minNutrition, maxNutrition, foods, cost, nutritionValues)
```

The first statement imports the *dietmodel module*, which must be stored in file *dietmodel.py* in the current directory. The second passes the model data to the `solve` function in the newly imported module.

1.1.10 Callbacks

Examples

callback

The final example we consider is the *callback example*, which demonstrates the use of Gurobi callbacks. Callbacks are used to report on the progress of the optimization or to modify the behavior of the Gurobi solver. To use a callback, the user writes a routine that implements the desired behavior. The routine is passed to the Gurobi Optimizer when optimization begins, and the routine is called regularly during the optimization process. One argument of the user routine is a `where` value, which indicates from where in the optimization process the callback is invoked. The user callback routine can call the optimization library to query certain values. We refer the reader to the callback section of the [Gurobi Reference Manual](#) for more precise details.

Our callback example implements a simple termination scheme: the user passes a node count (`limit`) into the callback, and the callback asks the optimizer to terminate when that node count is reached. This is implemented as follows:

C

C++

C#

Java

Python

Visual Basic

```
double nodecnt;
GRBcbget(cbdata, where, GRB_CB_MIP_NODCNT, &nodecnt);
if (nodecnt > limit)
    GRBterminate(model);
```

```
double nodecnt = getDoubleInfo(GRB_CB_MIP_NODCNT);
if (nodecnt > limit)
    abort();
```

```
double nodecnt = GetDoubleInfo(GRB.Callback.MIP_NODCNT);
if (nodecnt > limit)
    Abort();
```

```
double nodecnt = getDoubleInfo(GRB, CB_MIP_NODCNT);
if (nodecnt > limit)
    abort();
```

```
nodecnt = model.cbGet(GRB.Callback.MIP_NODCNT)
if nodecnt > model._myodelimit:
    model.terminate()
```

```
Dim nodecnt As Double = GetDoubleInfo(GRB.Callback.MIP_NODCNT)
If nodecnt > limit Then
    Abort()
End If
```

Our *callback example* also prints progress information.

C

C++

C#

Java

Python

Visual Basic

```
GRBcbget(cbdata, where, GRB_CB_MIP_NODCNT, &nodecnt);
if (nodecnt - mydata->lastmsg >= 100) {
    ...
    printf("%7.0f ...", nodecnt, ...);
}
```

```
double nodecnt = getDoubleInfo(GRB_CB_MIP_NODCNT);
if (nodecnt - lastnode >= 100) {
    lastnode = nodecnt;
    ...
    cout << nodecnt << " " << ...;
}
```

```
double nodecnt = GetDoubleInfo(GRB.Callback.MIP_NODCNT);
if (nodecnt - lastnode >= 100) {
    lastnode = nodecnt;
    ...
    Console.WriteLine(nodecnt + " " ...);
}
```

```
double nodecnt = getDoubleInfo(GRB.CB_MIP_NODCNT);
if (nodecnt - lastnode >= 100) {
    lastnode = nodecnt;
    ...
    System.out.println(nodecnt + " " ...);
}
```

```
nodecnt = model.cbGet(GRB.Callback.MIP_NODCNT)
if nodecnt % 100 == 0:
    print(f'{nodecnt} ...')
```

```
Dim nodecnt As Double = GetDoubleInfo(GRB.Callback.MIP_NODCNT)
If nodecnt - lastnode >= 100 Then
    lastnode = nodecnt
    ...
    Console.WriteLine(nodecnt & " " ...)
End If
```

